# GeoEngage Backend

## Technical Documentation

Version 1.0 | 2025

# 1. Workflow and Architecture

## 1.1 Request Flow

- **1.** Request → CORS Middleware → SlowAPI (rate limiter) → Route handler.
- **2.** Protected routes: `Authorization: Bearer <Firebase ID token>` → `get_current_user` (Firebase verify → `get_or_create_user`) or `get_current_admin`.
- **3.** Route obtains a DB session via `get_db()` and delegates to the service layer.
- **4.** Service returns a result; route returns HTTP response or raises `HTTPException` / validation error.
- **5.** Any unhandled exception → generic exception handler → 500 JSON response.

## 1.2 Zone Entry Flow

Client sends `POST /api/v1/event` with `event_type=zone_entry` (or `"zone"`) plus `zone_id` or `(zone_name, floor_id)`. The backend then:

- Resolves the zone by UUID or name + floor.
- Inserts an `Event` row for analytics.
- Upserts `UserZoneSession` — sets current zone and `has_transaction=False`.
- Finds the active `zone_entry` campaign for the zone (if any).
- Sends FCM push with data payload (campaign_id, notification_id, zone_name, type).
- Inserts `Notification` row (status: `sent` or `failed`).
- Returns: `success`, `notification_sent`, `campaign_message` or `message`.

## 1.3 Zone Exit (No Transaction) Flow

Client sends `POST /api/v1/event` with `event_type=zone_exit`. The backend then:

- Resolves the zone.
- Fetches `UserZoneSession` for the user.
- If no session or session zone differs from requested zone → returns early (no action).
- If `session.has_transaction = True` → deletes session and returns (exit campaign suppressed).
- Otherwise: finds active `zone_exit_no_txn` campaign → sends FCM → inserts `Notification` → deletes session.
- Returns: `success`, `notification_sent`, campaign message or status message.

## 1.4 Transaction Flow

Client sends `POST /api/v1/transactions` with `zone_id` or `(zone_name, floor_id)`. The backend:

- Resolves the zone.
- Inserts a `Transaction` row.
- If a `UserZoneSession` exists for the user in the same zone, sets `has_transaction=True`.
- A subsequent zone-exit call will detect this flag and skip the exit campaign.

## 1.5 Component Overview

| Component | Role |
|---|---|
| Clients (Android / Web) | Users and admins communicate via HTTP + Bearer token. |
| FastAPI App (/api/v1) | Routers: auth, events, campaigns, notifications, analytics, zones, floors, transactions. |
| Auth (deps) | Firebase verify_id_token → get_or_create_user / get_current_admin. |
| Services | user, event, campaign, notification, analytics, transaction — use DB session + FCM. |
| Database | SQLAlchemy engine (pooled) → PostgreSQL (Supabase). |
| Push (FCM) | Firebase Cloud Messaging via send_fcm_to_token. |

## 2. File Structure

```
GeoEngage/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── exceptions.py
│   ├── api/
│   │   ├── deps.py
│   │   └── v1/
│   │       ├── router.py
│   │       ├── auth.py
│   │       ├── events.py
│   │       ├── campaigns.py
│   │       ├── notifications.py
│   │       ├── analytics.py
│   │       ├── zones.py
│   │       ├── floors.py
│   │       └── transactions.py
│   ├── core/
│   │   ├── firebase.py
│   │   ├── fcm.py
│   │   └── rate_limit.py
│   ├── db/
│   │   ├── base.py
│   │   ├── session.py
│   │   └── models/
│   │       ├── __init__.py
│   │       ├── user.py
│   │       ├── floor.py
│   │       ├── zone.py
│   │       ├── campaign.py
│   │       ├── event.py
│   │       ├── notification.py
│   │       ├── transaction.py
│   │       └── user_zone_session.py
│   ├── schemas/
│   │   ├── user.py
│   │   ├── event.py
│   │   ├── campaign.py
│   │   ├── notification.py
│   │   ├── analytics.py
│   │   └── transaction.py
│   └── services/
│       ├── user.py
│       ├── event.py
│       ├── campaign.py
│       ├── notification.py
│       ├── analytics.py
│       └── transaction.py
├── scripts/
│   └── seed_supabase.py
├── tests/
│   ├── conftest.py
│   └── api/
│       ├── test_events.py
│       ├── test_campaigns.py
│       ├── test_analytics.py
│       └── test_notifications.py
├── .env.example
├── .gitignore
├── plan.md
├── README.md
└── requirements.txt
```

# 3. Features → API Mapping

| Feature | API |
|---|---|
| Health check | GET /health |
| Current user profile | GET /api/v1/me |
| Register / update FCM token | POST /api/v1/register-device |
| Verify admin (web dashboard) | GET /api/v1/verify-admin |
| Floor entry — welcome notification | POST /api/v1/event (event_type=floor, floor_id) |
| Zone entry — record event + optional campaign push | POST /api/v1/event (event_type=zone_entry or zone, zone_id or zone_name+floor_id) |
| Zone exit — optional exit-without-transaction push | POST /api/v1/event (event_type=zone_exit, zone_id or zone_name+floor_id) |
| Record transaction in zone | POST /api/v1/transactions |
| Notification history (paginated) | GET /api/v1/notifications |
| Record notification click for CTR | POST /api/v1/notification-click |
| List zones (optionally by floor) | GET /api/v1/zones (?floor_id) |
| List floors | GET /api/v1/floors |
| Create campaign | POST /api/v1/campaigns |
| Activate or deactivate campaign | PUT /api/v1/campaigns/{id} |
| List campaigns (with optional filters) | GET /api/v1/campaigns (?zone_id, ?trigger) |
| Delete campaign | DELETE /api/v1/campaigns/{id} |
| Dashboard analytics | GET /api/v1/analytics |

# 4. Database Schema

Hosted on PostgreSQL (Supabase). All timestamps are TIMESTAMPTZ.

## 4.1 users

| Column | Type | Constraints |
|---|---|---|
| id | SERIAL | PRIMARY KEY |
| firebase_uid | VARCHAR(128) | UNIQUE, NOT NULL |
| email | VARCHAR(255) | NOT NULL |
| name | VARCHAR(255) | NULL |
| role | VARCHAR(20) | NOT NULL, DEFAULT 'user', CHECK (user / admin) |
| fcm_token | TEXT | NULL |
| created_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |

## 4.2 floors

| Column | Type | Constraints |
|---|---|---|
| floor_id | INTEGER | PRIMARY KEY |
| floor_name | VARCHAR(255) | NOT NULL |

## 4.3 zones

| Column | Type | Constraints |
|---|---|---|
| id | UUID | PRIMARY KEY |
| floor_id | INTEGER | NOT NULL, FK → floors.floor_id CASCADE |
| name | VARCHAR(100) | NOT NULL |
| created_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |

Unique constraint: (floor_id, name).

## 4.4 campaigns

| Column | Type | Constraints |
|---|---|---|
| id | SERIAL | PRIMARY KEY |
| zone_id | UUID | NOT NULL, FK → zones.id CASCADE |
| name | VARCHAR(255) | NULL |
| message | TEXT | NOT NULL |
| trigger | VARCHAR(50) | NOT NULL, DEFAULT 'zone_entry' |

| Column | Type | Constraints |
|---|---|---|
| active | BOOLEAN | NOT NULL, DEFAULT FALSE |
| created_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |

## 4.5 events

| Column | Type | Constraints |
|---|---|---|
| id | SERIAL | PRIMARY KEY |
| user_id | INTEGER | NOT NULL, FK → users.id CASCADE |
| zone_id | UUID | NOT NULL, FK → zones.id CASCADE |
| entered_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |

## 4.6 notifications

| Column | Type | Constraints |
|---|---|---|
| id | SERIAL | PRIMARY KEY |
| user_id | INTEGER | NOT NULL, FK → users.id CASCADE |
| campaign_id | INTEGER | NOT NULL, FK → campaigns.id CASCADE |
| status | VARCHAR(20) | NOT NULL, CHECK (sent / failed) |
| fcm_message_id | TEXT | NULL |
| created_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |
| clicked_at | TIMESTAMPTZ | NULL |

## 4.7 transactions

| Column | Type | Constraints |
|---|---|---|
| id | SERIAL | PRIMARY KEY |
| user_id | INTEGER | NOT NULL, FK → users.id CASCADE |
| zone_id | UUID | NOT NULL, FK → zones.id CASCADE |
| created_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |

## 4.8 user_zone_sessions

| Column | Type | Constraints |
|---|---|---|
| id | SERIAL | PRIMARY KEY |
| user_id | INTEGER | NOT NULL, UNIQUE, FK → users.id CASCADE |
| zone_id | UUID | NOT NULL, FK → zones.id CASCADE |
| entered_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |
| has_transaction | BOOLEAN | NOT NULL, DEFAULT FALSE |

| Column | Type | Constraints |
|---|---|---|
| updated_at | TIMESTAMPTZ | NOT NULL, DEFAULT NOW() |

# 5. Pydantic Models (Schemas)

## user.py

- **MeResponse:** id, email, name, role, firebase_uid.
- **RegisterDeviceRequest:** fcm_token.

## event.py

- **EventRequest:** event_type, zone_id, zone_name, floor_id.
- Validation: `floor` requires floor_id; `zone_entry` / `zone_exit` require zone_id or (zone_name + floor_id).

## campaign.py

- **CampaignCreate:** zone_id (UUID str), message, name?, trigger?
- **CampaignUpdate:** active (bool).
- **CampaignResponse:** id, zone_id, message, name, active, trigger, created_at.

## notification.py

- **NotificationClickRequest:** campaign_id.

## analytics.py

- **ZoneEntryItem:** zone, count.
- **TopZoneItem:** zone_name, ctr.
- **AnalyticsResponse:** zone_entries, notifications_sent, clicks, ctr, top_zones.

## transaction.py

- **TransactionCreate:** zone_id, zone_name, floor_id.
- Validation: zone_id or (zone_name + floor_id) is required.

# 6. Error Handling

## 6.1 Global Handlers (app/main.py)

| Exception | HTTP Status | Response Body |
|---|---|---|
| HTTPException | exc.status_code | { detail, status_code } |
| RequestValidationError | 422 | { detail (concatenated), status_code: 422, errors } |
| Exception (unhandled) | 500 | { detail: "Internal server error", status_code: 500 } |
| RateLimitExceeded | 429 | SlowAPI default response |

## 6.2 Route-Level Responses

| Status | Cause |
|---|---|
| 401 Unauthorized | Invalid or missing Bearer token (get_current_user). |
| 403 Forbidden | Non-admin calling admin-only route or verify-admin. |
| 404 Not Found | Zone, floor, or campaign not found (events, campaigns, transactions). |
| 422 Unprocessable Entity | Invalid zone_id UUID, invalid event_type, or Pydantic validation failure. |
| 500 Internal Server Error | User sync failure or uncaught exception (events route logs and re-raises). |

## 6.3 Service-Level Behaviour

- FCM sends failures, sets notification `status` to `"failed"` and does not fail the HTTP request — delivery is best-effort.
- Zone resolution returning `None` causes the route to raise 404; it does not propagate as a 500.

# 7. Performance Testing — Apache JMeter

The GeoEngage backend was subjected to load testing using Apache JMeter 5.6.3 to validate API performance under concurrent user conditions.

## 7.1 Test Configuration

The following Thread Group parameters were applied for the load test:

| Parameter | Value |
|---|---|
| Tool | Apache JMeter 5.6.3 |
| Number of Threads (Users) | 50 |
| Ramp-up Period | 0 seconds (all threads start simultaneously) |
| Loop Count | 1 |
| Same User on Each Iteration | Enabled |
| Delay Thread Creation Until Needed | Disabled |
| Action on Sampler Error | Continue |
| Target Endpoint | HTTP Request (GeoEngage API) |

## Thread Group Configuration Screenshot

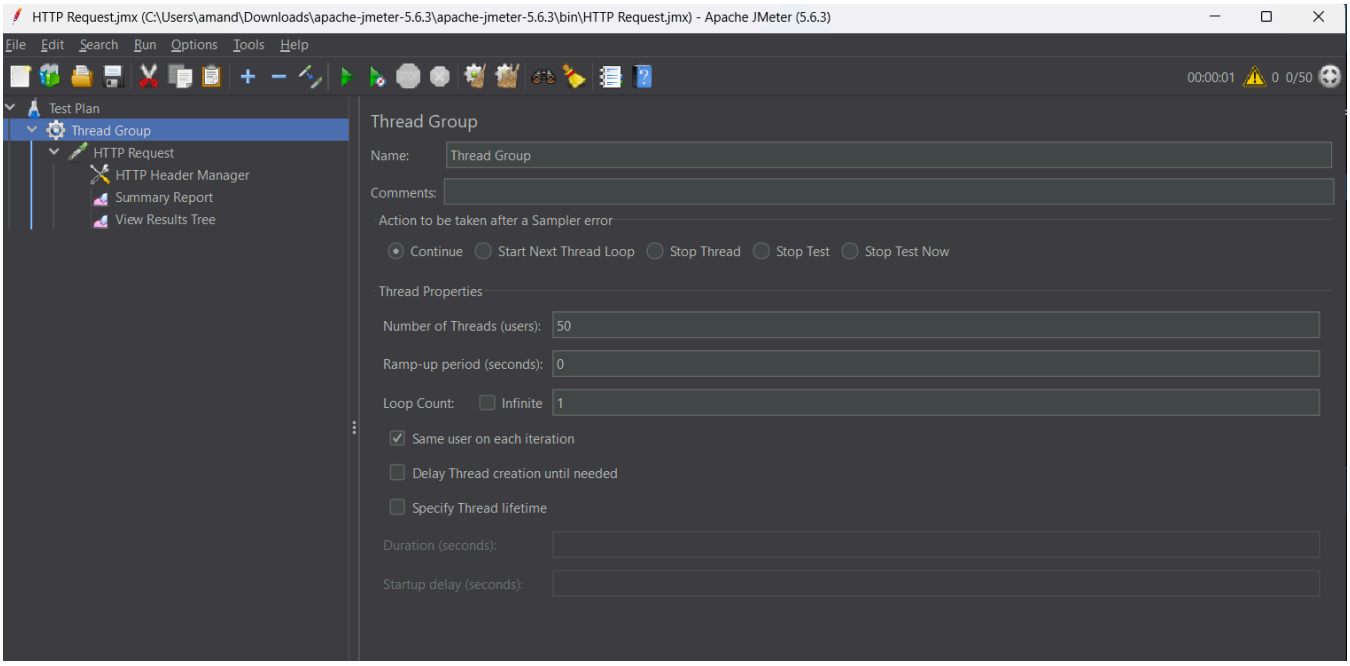


*Figure 1: JMeter Thread Group Configuration — 50 concurrent users, ramp-up 0s, loop count 1*

## 7.2 Test Results — Summary Report

The following metrics were recorded in the JMeter Summary Report after executing the test plan with 50 concurrent users:

| Label | # Samples | Avg (ms) | Min (ms) | Max (ms) | Std. Dev. | Error % | Throughput | Avg Bytes |
|---|---|---|---|---|---|---|---|---|
| HTTP Request | 50 | 725 | 478 | 952 | 169.75 | 0.00 % | 52.0/sec | 252.0 |
| **TOTAL** | **50** | **725** | **478** | **952** | **169.75** | **0.00 %** | **52.0/sec** | **252.0** |

## Summary Report Screenshot

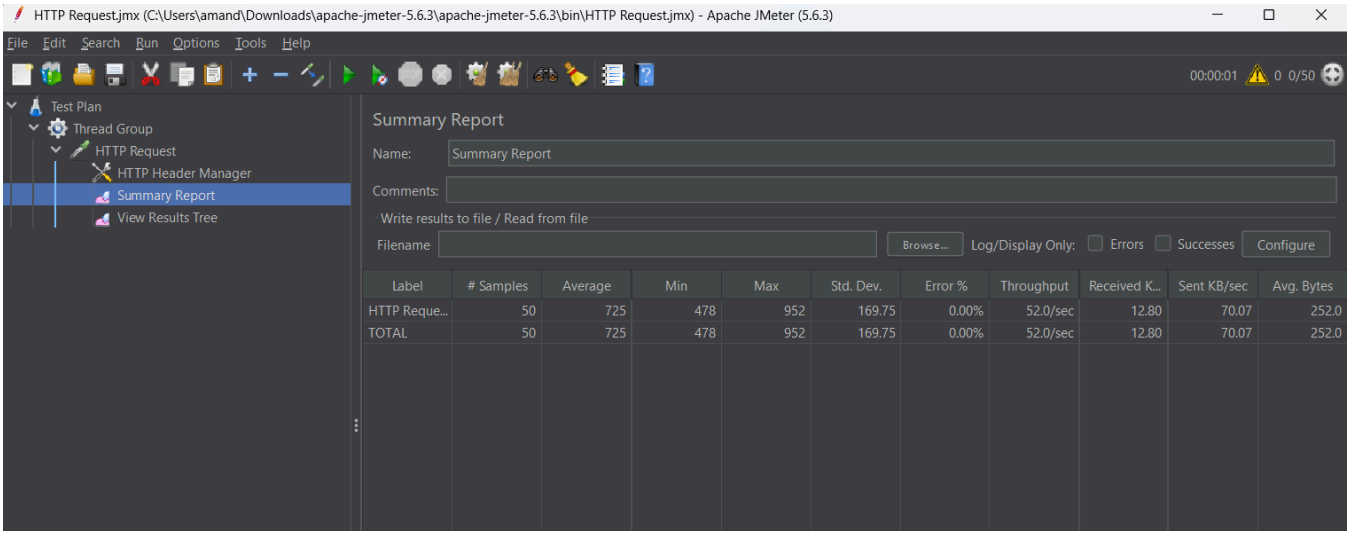


*Figure 2: JMeter Summary Report — Results for 50 concurrent users*

## 7.3 Performance Analysis

| Metric | Value | Assessment |
|---|---|---|
| Average Response Time | 725 ms | Acceptable for authenticated API with DB + FCM calls |
| Minimum Response Time | 478 ms | Best-case latency under concurrent load |
| Maximum Response Time | 952 ms | Worst-case latency; within sub-second threshold |
| Standard Deviation | 169.75 ms | Moderate variance; some contention under full concurrency |
| Error Rate | 0.00% | All 50 requests succeeded — no failures observed |
| Throughput | 52.0 req/sec | Healthy throughput for 50 simultaneous users |
| Avg Response Size | 252 bytes | Lightweight JSON payloads confirmed |

## 7.4 Key Observations

- Zero errors recorded across all 50 concurrent requests, confirming API stability under load.
- Average response time of 725 ms is within acceptable range for an authenticated endpoint involving Firebase token verification, PostgreSQL queries, and optional FCM dispatch.
- Throughput of 52.0 requests/second demonstrates the FastAPI + SQLAlchemy stack handles concurrency effectively.
- The 474 ms spread between min and max response times (std. dev. 169.75 ms) suggests mild contention at the database connection pool or FCM layer under simultaneous load.

- All requests returned an average of 252 bytes, consistent with lightweight JSON success responses.